

**Московский авиационный институт
(национальный исследовательский университет)**

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №8 по курсу «Информационный поиск»

Студент: М.А. Пирязев
Преподаватель: А.А. Кухтичев
Группа: М8О-407Б-22
Дата: 21.12.2025
Оценка:
Подпись:

Москва, 2025

Оглавление

1. Лабораторная работа №8 “Ранжирование TF-IDF”	2
1.1 Описание	2
2. Исходный код.....	2
2.1 Используемые данные и входные файлы	2
2.2 Общая схема решения	2
2.3 Ранжирование TF-IDF	3
2.4 Примеры выполнения поисковых запросов	4
2.5 Сборка и запуск.....	5
2.6 Тестирование	5
2.7 Примеры неудачных запросов	12
2.8 Интерпретация числа score в выдаче и способ его вычисления	13
3. Выводы	16
Список литературы.....	17

1. Лабораторная работа №8 “Ранжирование TF-IDF”

1.1 Описание

Целью работы является построение ранжированного поиска по коллекции документов с использованием статистической меры TF-IDF, а также обеспечение корректной обработки двух классов запросов: нечётких запросов (набор слов) и булевых запросов (логическое выражение с операторами AND/OR/NOT и скобками).

2. Исходный код

2.1 Используемые данные и входные файлы

В качестве источника данных используется корпус документов, ранее загруженный и нормализованный в рамках предыдущих лабораторных работ. Для выполнения ранжированного поиска применяются два файла индекса формата Lab6: - прямой индекс `index6.fwd`, содержащий метаданные документа (URL и заголовок) и обеспечивающий восстановление информации по docid; - инвертированный индекс `index6.inv` (формат INV2), содержащий словарь терминов и постинговые списки. Постинговые списки хранятся в сжатом виде (d-gap по docid и variable-byte кодирование). Для каждого термина в постинге присутствует частота *tf* (количество вхождений термина в документ), а также позиционные данные; при ранжировании используются *tf* и документная частота *df*, позиционные данные не используются.

2.2 Общая схема решения

Программа `rank_search8` выполняет следующие этапы:

- открывает файлы индекса `index6.inv` и `index6.fwd` и проверяет согласованность числа документов;
- разбирает пользовательский запрос, определяя, является ли он булевым (наличие операторов `&&`, `||`, `!` или скобок) либо нечётким;

- формирует множество кандидатов;
- для нечёткого запроса: объединение (OR) постинговых списков всех термов запроса;
- для булевого запроса: вычисление результата логического выражения над постинговыми списками (через перевод в ОПН и стековый вычислитель);
- вычисляет TF-IDF скор для каждого документа-кандидата и сортирует результаты по убыванию сора;
- выводит общее число найденных документов (hits), а также top-N документов с указанием docid, значения score, заголовка и URL.}

2.3 Ранжирование TF-IDF

Ранжирование реализовано на основе классической схемы TF-IDF с использованием документной частоты df , вычисляемой по инвертированному индексу.

Обозначения:

N - число документов в коллекции;

$tf(t, d)$ - частота термина t в документе d (из постингового списка);

$df(t)$ - число документов, содержащих терм t (равно длине постингового списка по терму);

$idf(t)$ - обратная документная частота, вычисляемая по формуле

$$idf(t) = \log((N + 1) / (df(t) + 1)).$$

Итоговый скор документа d для запроса q вычисляется как сумма вкладов по термам:

$$score(d, q) = \sum_{t \in q} tf(t, d) * idf(t).$$

Для обеспечения устойчивости вычислений применяется сглаживание (+1) в числителе и знаменателе, что исключает деление на ноль и корректно обрабатывает крайние случаи.

2.4 Примеры выполнения поисковых запросов

Ниже приведён пример выполнения нечёткого запроса (термы через пробел). Программа выводит количество найденных документов и top-5 документов, отсортированных по TF-IDF скору:

```
./build/rank_search8 --inv out6/index6.inv --fwd out6/index6.fwd --q "дед мороз" --top 3
```

Далее выводятся строки формата: docID score title url

hits 1013

34418 459.031 Дед Мороз

https://ru.wikipedia.org/wiki/%D0%94%D0%B5%D0%B4_%D0%9C%D0%BE%D1%80%D0%BE%D0%B7

8943 146.149 Зимние фольклорные персонажи

https://ru.wikipedia.org/wiki/%D0%97%D0%B8%D0%BC%D0%BD%D0%B8%D0%B5_%D1%84%D0%BE%D0%BB%D1%8C%D0%BA%D0%BB%D0%BE%D1%80%D0%BD%D1%8B%D0%B5_%D0%BF%D0%B5%D1%80%D1%81%D0%BE%D0%BD%D0%B0%D0%B6%D0%B8

35600 137.054 Мороз, Александр Александрович

https://ru.wikipedia.org/wiki/%D0%9C%D0%BE%D1%80%D0%BE%D0%B7%2C_%D0%90%D0%BB%D0%B5%D0%BA%D1%81%D0%B0%D0%BD%D0%B4%D1%80_%D0%90%D0%BB%D0%B5%D0%BA%D1%81%D0%B0%D0%BD%D0%B4%D1%80%D0%BE%D0%B2%D0%B8%D1%87

time_ms 5

Данный пример демонстрирует, что на первой позиции оказывается документ, наиболее полно соответствующий теме запроса, а далее следуют документы, где встречаются отдельные термины запроса либо связанные понятия. Для булевого запроса пример имеет вид:

```
./build/rank_search8 --inv out6/index6.inv --fwd out6/index6.fwd --q "дед && мороз" --top 3
```

hits 39

34418 459.031 Дед Мороз

https://ru.wikipedia.org/wiki/%D0%94%D0%B5%D0%B4_%D0%9C%D0%BE%D1%80%D0%BE%D0%B7

8943 146.149 Зимние фольклорные персонажи

https://ru.wikipedia.org/wiki/%D0%97%D0%B8%D0%BC%D0%BD%D0%B8%D0%B5_%D1%84%D0%BE%D0%BB%D1%8C%D0%BA%D0%BB%D0%BE%D1%80%D0%BD%D1%8B%D0%B5_%D0%BF%D0%B5%D1%80%D1%81%D0%BE%D0%BD%D0%B0%D0%B6%D0%B8

29817 31.4975 Папанов, Анатолий Дмитриевич

https://ru.wikipedia.org/wiki/%D0%9F%D0%B0%D0%BF%D0%B0%D0%BD%D0%BE%D0%B2%2C_%D0%90%D0%BD%D0%B0%D1%82%D0%BE%D0%BB%D0%B8%D0%B9_%D0%94%D0%BC%D0%B8%D1%82%D1%80%D0%B8%D0%B5%D0%B2%D0%B8%D1%87

<https://ru.wikipedia.org/wiki/%D0%9C%D0%B8%D1%85%D0%B0%D0%B9%D0%BB%D0%BE%D0%B2%D0%BD%D0%B0>

time_ms 4

В этом случае условие соответствия является строгим: в результат попадают только документы, содержащие оба термина, однако порядок выдачи по-прежнему определяется TF-IDF.

2.5 Сборка и запуск

Сборка C++-части проекта выполняется через CMake:

```
cd cpp/boolean  
cmake -S . -B build  
cmake --build build -j
```

Запуск программы осуществляется командой:

```
./build/rank_search8 --inv <path_to_index6.inv> --fwd <path_to_index6.fwd> --q  
"<query>" --top <N>
```

Также поддерживается режим запуска набора запросов из файла:

```
./build/rank_search8 --inv <...> --fwd <...> --qfile queries.txt --top <N>
```

2.6 Тестирование

Для проверки корректности ключевых компонентов реализованы модульные тесты на основе фреймворка Catch2 и интеграции с CTest.

В тестах проверяются:

- корректность токенизации и приведение к нижнему регистру;
- поведение опции `keep_numbers`;
- корректность булевых операций AND/OR/NOT над отсортированными списками docID;
- корректность построения ОПН (RPN) для простых булевых выражений;
- базовый smoke-тест на открытие индекса INV2 (обработка отсутствующего файла без аварийного завершения).

Запуск тестов:

cd tests

cmake -S . -B build

cmake --build build -j

ctest --test-dir build -V

Вывод программы после прохождения тестирования:

```
michael@Huawei:~/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests$ ctest --test-dir build -V
Internal ctest changing into directory: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/build
UpdateCTestConfiguration from :/home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/build/DartConfiguration.tcl
Parse Config file:/home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/build/DartConfiguration.tcl
UpdateCTestConfiguration from :/home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/build/DartConfiguration.tcl
Parse Config file:/home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/build/DartConfiguration.tcl
Test project /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/build
Constructing a list of tests
Done constructing a list of tests
Updating test list for fixtures
Added 0 tests to meet fixture requirements
Checking test dependency graph...
Checking test dependency graph end
test 1
  Start 1: Tokenizer: basic lowercase

1: Test command: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/build/unit_tests "Tokenizer: basic lowercase" "-s"
1: Test timeout computed to be: 1500
1: Filters: "Tokenizer: basic lowercase"
1: Randomness seeded to: 986910278
1:
1: ~~~~~
1: unit_tests is a Catch2 v3.5.2 host application.
1: Run with -? for options
1:
1: -----
1: Tokenizer: basic lowercase
1: -----
1: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:34
1: .....
1:
1: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:38: PASSED:
1: REQUIRE( out.size() == 3 )
1: with expansion:
1: 3 == 3
1:
1: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:39: PASSED:
1: REQUIRE( out[0] == "python" )
1: with expansion:
```

```

1: "python" == "python"
1:
1: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:40: PASSED:
1: REQUIRE( out[1] == "н" )
1: with expansion:
1: "н" == "н"
1:
1: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:41: PASSED:
1: REQUIRE( out[2] == "java" )
1: with expansion:
1: "java" == "java"
1:
1: =====
1: All tests passed (4 assertions in 1 test case)
1:
1/5 Test #1: Tokenizer: basic lowercase ..... Passed 0.01 sec
test 2
    Start 2: Tokenizer: keep_numbers

2: Test command: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/build/unit_tests "Tokenizer:
keep_numbers" "-s"
2: Test timeout computed to be: 1500
2: Filters: "Tokenizer: keep_numbers"
2: Randomness seeded to: 52718554
2:
2: ~~~~~
2: unit_tests is a Catch2 v3.5.2 host application.
2: Run with -? for options
2:
2: -----
2: Tokenizer: keep_numbers
2: keep_numbers = false
2: -----
2: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:47
2: .....
2:
2: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:51: PASSED:
2: REQUIRE( out.size() == 2 )
2: with expansion:
2: 2 == 2
2:
2: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:52: PASSED:
2: REQUIRE( out[0] == "win" )
2: with expansion:
2: "win" == "win"
2:
2: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:53: PASSED:
2: REQUIRE( out[1] == "abc" )
2: with expansion:
2: "abc" == "abc"
2:
2: -----
2: Tokenizer: keep_numbers

```



```

2: keep_numbers = true
2: -----
2: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:56
2: .....
2:
2: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:60: PASSED:
2: REQUIRE( out.size() == 3 )
2: with expansion:
2: 3 == 3
2:
2: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:61: PASSED:
2: REQUIRE( out[0] == "win10" )
2: with expansion:
2: "win10" == "win10"
2:
2: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:62: PASSED:
2: REQUIRE( out[1] == "123" )
2: with expansion:
2: "123" == "123"
2:
2: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:63: PASSED:
2: REQUIRE( out[2] == "abc123" )
2: with expansion:
2: "abc123" == "abc123"
2:
2: =====
2: All tests passed (7 assertions in 1 test case)
2:
2/5 Test #2: Tokenizer: keep_numbers ..... Passed    0.01 sec
test 3
    Start 3: Boolean ops: AND/OR/NOT

3: Test command: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/build/unit_tests "Boolean ops:
AND/OR/NOT" "-s"
3: Test timeout computed to be: 1500
3: Filters: "Boolean ops: AND/OR/NOT"
3: Randomness seeded to: 3680058606
3:
3: ~~~~~
3: unit_tests is a Catch2 v3.5.2 host application.
3: Run with -? for options
3:
3: -----
3: Boolean ops: AND/OR/NOT
3: AND
3: -----
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:68
3: .....
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:73: PASSED:
3: REQUIRE( r.size == 2 )
3: with expansion:
3: 2 == 2

```

```

3:
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:74: PASSED:
3: REQUIRE( r.data[0] == 2u )
3: with expansion:
3: 2 == 2
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:75: PASSED:
3: REQUIRE( r.data[1] == 4u )
3: with expansion:
3: 4 == 4
3:
3: -----
3: Boolean ops: AND/OR/NOT
3: OR
3: -----
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:78
3: .....
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:83: PASSED:
3: REQUIRE( r.size == 5 )
3: with expansion:
3: 5 == 5
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:84: PASSED:
3: REQUIRE( r.data[0] == 1u )
3: with expansion:
3: 1 == 1
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:85: PASSED:
3: REQUIRE( r.data[1] == 2u )
3: with expansion:
3: 2 == 2
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:86: PASSED:
3: REQUIRE( r.data[2] == 3u )
3: with expansion:
3: 3 == 3
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:87: PASSED:
3: REQUIRE( r.data[3] == 4u )
3: with expansion:
3: 4 == 4
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:88: PASSED:
3: REQUIRE( r.data[4] == 5u )
3: with expansion:
3: 5 == 5
3:
3: -----
3: Boolean ops: AND/OR/NOT
3: NOT
3: -----
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:91

```

```

3: .....
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:95: PASSED:
3: REQUIRE( r.size == 3 )
3: with expansion:
3: 3 == 3
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:96: PASSED:
3: REQUIRE( r.data[0] == 0u )
3: with expansion:
3: 0 == 0
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:97: PASSED:
3: REQUIRE( r.data[1] == 2u )
3: with expansion:
3: 2 == 2
3:
3: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:98: PASSED:
3: REQUIRE( r.data[2] == 4u )
3: with expansion:
3: 4 == 4
3:
3: =====
3: All tests passed (13 assertions in 1 test case)
3:
3/5 Test #3: Boolean ops: AND/OR/NOT ..... Passed    0.01 sec
test 4
    Start 4: Query parser: RPN for simple AND

4: Test command: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/build/unit_tests "Query parser: RPN
for simple AND" "-s"
4: Test timeout computed to be: 1500
4: Filters: "Query parser: RPN for simple AND"
4: Randomness seeded to: 2045547624
4:
4: ~~~~~
4: unit_tests is a Catch2 v3.5.2 host application.
4: Run with -? for options
4:
4: -----
4: Query parser: RPN for simple AND
4: -----
4: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:102
4: .....
4:
4: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:106: PASSED:
4: REQUIRE( rpn.size == 3 )
4: with expansion:
4: 3 == 3
4:
4: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:107: PASSED:
4: REQUIRE( rpn.data[0].type == QTokType::TERM )
4: with expansion:

```

```

4: 0 == 0
4:
4: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:108: PASSED:
4: REQUIRE( rpn.data[1].type == QTokenType::TERM )
4: with expansion:
4: 0 == 0
4:
4: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:109: PASSED:
4: REQUIRE( rpn.data[2].type == QTokenType::AND )
4: with expansion:
4: 1 == 1
4:
4: =====
4: All tests passed (4 assertions in 1 test case)
4:
4/5 Test #4: Query parser: RPN for simple AND ..... Passed    0.01 sec
test 5
    Start 5: INV2: open missing file returns false

5: Test command: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/build/unit_tests "INV2: open
missing file returns false" "-s"
5: Test timeout computed to be: 1500
5: Filters: "INV2: open missing file returns false"
5: Randomness seeded to: 2433235896
5:
5: ~~~~~
5: unit_tests is a Catch2 v3.5.2 host application.
5: Run with -? for options
5:
5: -----
5: INV2: open missing file returns false
5: -----
5: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:112
5: .....
5:
5: /home/michael/InformationSearch/SearchEngine/Lab_8_TF-IDF/tests/unit_tests.cpp:114: PASSED:
5: REQUIRE( inv.open("definitely_not_exists_12345.inv") == false )
5: with expansion:
5: false == false
5:
5: =====
5: All tests passed (1 assertion in 1 test case)
5:
5/5 Test #5: INV2: open missing file returns false ... Passed    0.01 sec
100% tests passed, 0 tests failed out of 5
Total Test time (real) = 0.04 sec

```

2.7 Примеры неудачных запросов

В данном разделе приведены примеры выполнения нескольких запросов. В качестве «неудачных» рассматриваются случаи, когда запрос является слишком общим либо многозначным; в таких ситуациях TF-IDF корректно ранжирует документы по статистическим признакам (частота и редкость терминов), однако не способен разрешать семантическую неоднозначность запроса.

`./build/rank_search8 --inv out6/index6.inv --fwd out6/index6.fwd --q "мир" --top 5`

```
michael@Huawei:~/InformationSearch/SearchEngine/Lab_8_TF-IDF/cpp/boolean$ ./build/rank_search8 --inv
out6/index6.inv --fwd out6/index6.fwd --q "мир" --top 5
hits 5595
5594 302.728 Мир https://ru.wikipedia.org/wiki/%D0%9C%D0%B8%D1%80
10755 175.871 Талышское ханство
https://ru.wikipedia.org/wiki/%D0%A2%D0%B0%D0%BB%D1%8B%D1%88%D1%81%D0%BA%D0%BE%D0%B5_%
D1%85%D0%B0%D0%BD%D1%81%D1%82%D0%B2%D0%BE
21971 155.689 Мир Полудня
https://ru.wikipedia.org/wiki/%D0%9C%D0%B8%D1%80_%D0%9F%D0%BE%D0%BB%D1%83%D0%B4%D0%BD%D
1%8F
14181 152.806 Новый мир (значения)
https://ru.wikipedia.org/wiki/%D0%9D%D0%BE%D0%B2%D1%8B%D0%B9_%D0%BC%D0%B8%D1%80_%28%D0%
B7%D0%BD%D0%B0%D1%87%D0%B5%D0%BD%D0%B8%D1%8F%29
3390 141.273 Война и мир
https://ru.wikipedia.org/wiki/%D0%92%D0%BE%D0%B9%D0%BD%D0%B0_%D0%B8_%D0%BC%D0%B8%D1%80
time_ms 7
```

Слово «мир» является многозначным: оно может означать «вселенную», «отсутствие войны», «общество», а также встречается в названиях множества объектов, организаций и произведений. Это подтверждается тем, что в Википедии существует отдельная страница значений «Мир», содержащая длинный перечень разных смыслов и объектов. В результате поиск по одному слову приводит к большому числу документов (hits 5595), а top-выдача включает статьи, относящиеся к разным тематикам (например, «Мир (значения)», «Мир Полудня», «Война и мир» и другие). Такое поведение является ожидаемым: TF-IDF не использует семантику и не умеет «угадывать», в каком смысле пользователь употребил слово, поэтому ранжирование определяется только статистикой встречаемости термина в документе.

`./build/rank_search8 --inv out6/index6.inv --fwd out6/index6.fwd --q "и" --top 5`

```
hits 75323
```

36829 293.016 Бхактиведанта Свами Прабхупада
https://ru.wikipedia.org/wiki/%D0%91%D1%85%D0%B0%D0%BA%D1%82%D0%B8%D0%B2%D0%B5%D0%B4%D0%B0%D0%BD%D1%82%D0%B0_%D0%A1%D0%B2%D0%B0%D0%BC%D0%B8_%D0%9F%D1%80%D0%B0%D0%B1%D1%85%D1%83%D0%BF%D0%B0%D0%B4%D0%B0
 15772 245.975 Структурализм
<https://ru.wikipedia.org/wiki/%D0%A1%D1%82%D1%80%D1%83%D0%BA%D1%82%D1%83%D1%80%D0%B0%D0%BB%D0%B8%D0%B7%D0%BC>
 16505 245.692 Украинская повстанческая армия
https://ru.wikipedia.org/wiki/%D0%A3%D0%BA%D1%80%D0%B0%D0%B8%D0%BD%D1%81%D0%BA%D0%B0%D1%8F_%D0%BF%D0%BE%D0%B2%D1%81%D1%82%D0%B0%D0%BD%D1%87%D0%B5%D1%81%D0%BA%D0%B0%D1%8F_%D0%B0%D1%80%D0%BC%D0%B8%D1%8F
 213 240.024 Мифология
<https://ru.wikipedia.org/wiki/%D0%9C%D0%B8%D1%84%D0%BE%D0%BB%D0%BE%D0%B3%D0%B8%D1%8F>
 9358 237.474 Война за испанское наследство
https://ru.wikipedia.org/wiki/%D0%92%D0%BE%D0%B9%D0%BD%D0%B0_%D0%B7%D0%B0_%D0%B8%D1%81%D0%BF%D0%B0%D0%BD%D1%81%D0%BA%D0%BE%D0%B5_%D0%BD%D0%B0%D1%81%D0%BB%D0%B5%D0%B4%D1%81%D1%82%D0%B2%D0%BE
 time_ms 62

Терм «и» является крайне частым служебным словом, встречающимся практически во всех текстах. Это приводит к очень большому числу документов в результате (hits 75323). В такой ситуации обратная документная частота *idf* для данного термина мала, и ранжирование становится слабо связанным с “содержательной” релевантностью: документы оказываются высоко в выдаче не потому, что они тематически соответствуют запросу (запрос не несёт темы), а потому что в них слово «и» встречается много раз. Таким образом, нечеткость данного запроса связана не с реализацией поиска, а с тем, что запрос не содержит информативных терминов.

2.8 Интерпретация числа *score* в выдаче и способ его вычисления

В выдаче программы рядом с каждым документом печатается вещественное число *score*. Это значение является мерой релевантности документа текущему запросу и используется для сортировки результатов: чем больше *score*, тем выше документ расположен в выдаче. В рамках данной лабораторной работы *score* вычисляется по схеме TF-IDF как сумма вкладов по всем терминам запроса.

При обработке запроса его текст сначала токенизируется тем же токенизатором, что и при построении индекса. Далее для каждого термина запроса из инвертированного индекса извлекается список постингов формата (*docID*, *tf*), где *docID* - идентификатор документа, а *tf* — число вхождений данного термина в документ.

Это значение `tf` уже хранится в индексе и не вычисляется повторно путём анализа исходных текстов.

Также используются следующие статистики:

`N` - общее число документов в коллекции, которое берётся из заголовка инвертированного индекса (`inv.docs()`).

`df` - документная частота термина, то есть число документов, где терм встретился хотя бы один раз. В реализации `df` вычисляется как длина списка постингов по данному терму: `df = plist.size`.

Чтобы уменьшить влияние слишком частых слов и, наоборот, усилить вклад редких терминов, используется обратная документная частота `idf`. В реализации применяется сглаженная формула:

$$idf = \log((N + 1) / (df + 1))$$

Сглаживание (+1 в числителе и знаменателе) необходимо для устойчивости вычислений и корректной обработки крайних случаев (например, если по каким-либо причинам терм отсутствует в индексе или если `df` близка к `N`). При фиксированном `N` величина `idf` тем больше, чем меньше `df`, то есть чем реже терм встречается в коллекции.

Фрагмент из `rank_search8_main.cpp`: вычисление TF-IDF score для документов-кандидатов.

```
const double N = static_cast<double>(inv.docs());
for (std::size_t ti = 0; ti < qterms.size; ti++) {
    nostl::Vec<PostingTF> plist;
    inv.get_postings_tf(qterms.data[ti], plist);
    const double df = static_cast<double>(plist.size);
    const double idf = std::log((N + 1.0) / (df + 1.0));
```

```

for (std::size_t j = 0; j < plist.size; j++) {
    const std::uint32_t docid = plist.data[j].docid;
    const std::uint32_t tf = plist.data[j].tf;
    std::uint32_t idxp = doc2idx.find(probe_docid_key(docid)); // docid -> индекс в массиве candidates
    if (!idxp) continue; // документ не является кандидатом
    scores.data[idxp] += static_cast<double>(tf) * idf; // вклад терма в итоговый score
}
}

```


3. Выводы

В ходе выполнения лабораторной работы №8 был реализован ранжированный поиск на основе схемы TF-IDF, что позволило упорядочивать найденные документы по степени релевантности запросу, а не выводить их в произвольном порядке. Были закреплены навыки работы с инвертированным индексом для извлечения статистик коллекции и вычисления итогового ранга документа как суммы вкладов по терминам запроса, учитывающей частоту термина в документе и его распространённость в коллекции.

Также были отработаны два режима обработки запросов: нечёткий режим для запросов, содержащих только термины через пробелы, и строгий режим для булевых запросов с операторами AND/OR/NOT. Показано, что даже при строгой фильтрации по булевым условиям результаты могут быть отсортированы по релевантности с использованием TF-IDF, что повышает практическую полезность выдачи.

Дополнительно были закреплены навыки тестирования и поддержки качества кода: подготовлены модульные тесты для базовых компонентов, что обеспечивает воспроизводимую проверку корректности и упрощает дальнейшее развитие проекта.

Список литературы

- [1] Маннинг, Рагхаван, Шютце *Введение в информационный поиск* — Издательский дом «Вильямс», 2011. Перевод с английского: доктор физ.-мат. наук Д.А. Ключина — 528 с. (ISBN 978-5-8459-1623-4 (рус.))
- [2] Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. 3-е изд. М.: ООО "И.Д. Вильямс", 2013.
- [3] Кнут Д. Э. Искусство программирования. Том 3. Сортировка и поиск. М.: ООО "И.Д. Вильямс", 2007.
- [4] Тененбаум Э., Лангсам И., Одженстейн М. Структуры данных с помощью С. М.: Вильямс, 2000.
- [5] Ватолин Д., Ратушняк А., Смирнов М., Юкин В. Методы сжатия данных. Устройство архиваторов, сжатие изображений и видео.